

CASE STUDY

TaskFlow AI

From Task Management to an AI-Powered Personal Productivity System

Project TaskFlow AI	Role Agentic AI Engineer / Python Backend Engineer
Category AI Productivity Platform	Core Stack Python · FastAPI · MongoDB · Beanie · Mistral Large

A repository-grounded product and engineering case study.

Repository: github.com/abizaid24/TaskFlow-AI-Backend

Project Overview

TaskFlow AI is a productivity backend designed around a broader idea: personal productivity should not be limited to creating and checking off tasks. The platform brings task management, health tracking, prayer tracking, notifications, analytics, and a conversational AI layer into one system.

The defining engineering decision is the AI agent. Rather than using an LLM only for conversation, TaskFlow AI gives the agent approved tools that can interact with real application capabilities. A user can describe an intention in natural language and the agent can invoke the appropriate backend operation.

The backend is built with FastAPI, MongoDB and Beanie, with an OpenAI Agent SDK pattern powered by Mistral Large. The repository also contains a service layer, repository layer, asynchronous data access, JWT authentication, background scheduling, notification integrations, analytics, Docker/Nginx infrastructure, and a 24-test suite.

Source basis: the supplied TaskFlow AI repository and its documented implementation. Product motivation is presented as the product rationale; implementation claims are limited to what the repository documents.

The Starting Point: Why Build Another To-Do App?

The project began with a familiar pattern in developer projects: build a basic To-Do application with task creation, reminders, priorities, notifications, and completion. These applications are useful for learning CRUD, authentication and API development, but the concept can remain narrow.

The product question behind TaskFlow AI was different: **What if a productivity application understood more of a person's day instead of only storing a list of tasks?**

That question led to a broader product direction—combine everyday task management with an AI companion, health tracking, prayer routines, reminders, progress signals, and a conversational interface capable of taking action.

Product Vision

TaskFlow AI was designed around the idea of a single intelligent productivity layer. Instead of switching between separate systems for tasks, health information, prayer tracking, notifications and progress, the user can interact with one application.

The vision is therefore not simply “a To-Do app with AI.” It is an AI-assisted personal productivity system where conversation becomes an additional interface to the underlying application.

PRODUCT PRINCIPLE

“Productivity should not be limited to checking boxes on a To-Do list.”

From Conversation to Action

The most important part of the architecture is the transition from natural-language intent to controlled backend action.

01 User intent	02 Agent interprets request	03 Approved tool call	04 Service + repository	05 MongoDB state
-------------------	--------------------------------	--------------------------	----------------------------	---------------------

"Create a task for tomorrow."	Mistral Large	create_task	Business logic	Persisted result
-------------------------------	---------------	-------------	----------------	------------------

This architecture keeps the language model responsible for interpreting intent while the application remains responsible for executing business operations. The repository documents tools for creating and completing tasks, retrieving today's and overdue tasks, logging water and steps, marking prayers complete, retrieving health summaries, and fetching notifications.

05 / CASE STUDY

The AI Agent Architecture

TaskFlow AI uses an OpenAI Agent SDK pattern with Mistral Large as the model. The agent sits behind a dedicated chat API and connects to application tools rather than directly manipulating database records.

Client Conversation	Agent API /api/v1/agent/chat	Agent Layer OpenAI Agent SDK pattern
Natural language	Session + request handling	Mistral Large + tool selection
Tools Task · Health · Prayer · Notifications	Service Layer Business logic	Repository / DB MongoDB + Beanie

Tool	Purpose
create_task	Create a new task from conversational intent.
complete_task	Mark an existing task as completed.
get_today_tasks / get_overdue_tasks	Retrieve relevant task state.
log_water_intake / log_steps	Record health activity.
mark_prayer_completed	Record completion of a prayer.
get_health_summary / get_notifications	Retrieve contextual user information.

06 / CASE STUDY

Task Management as a State Machine

A major engineering detail separates TaskFlow AI from a basic CRUD task API: task state is modeled as a deterministic finite-state machine (DFA).

PENDING	→	IN_PROGRESS	→	COMPLETED
↕		↕		→ ARCHIVED
CANCELLED → PENDING		COMPLETED → IN_PROGRESS (reopen)		ARCHIVED = terminal

The documented transitions allow tasks to move between pending, in-progress, completed, cancelled and archived states with explicit rules. This makes task behavior predictable and gives the API a stronger domain model than unrestricted status updates.

07 / CASE STUDY

Beyond Tasks: Health + Prayer

The broader product concept is reflected in the second intelligence layer: the backend tracks more than work items.

Health Tracking	Prayer Tracking	Progress Signals
Water, steps, calories and active minutes are represented in the documented feature set and health API.	Five daily prayers, completion state and streak logic are implemented in the documented backend.	Analytics include productivity scoring, streaks, achievements and motivational messaging.

08 / CASE STUDY

Automation: Keeping the Day in Sync

TaskFlow AI also moves beyond request/response APIs through scheduled background jobs using APScheduler. The repository documents recurring jobs for task reminders, prayer reminders, health digests, weekly reports, notification cleanup and missed-prayer handling.

Job	Schedule	Purpose
Task Reminders	Every minute	Due task reminders
Prayer Reminders	Hourly	Upcoming prayer reminders
Health Digest	Daily @ 9 PM	Daily health summary
Weekly Report	Sundays @ 8 PM	Weekly productivity report
Data Cleanup	Daily @ 3 AM	Old notification cleanup
Missed Prayers	Daily @ 9 PM	Mark prayers as missed

09 / CASE STUDY

Analytics, Streaks & Achievements

The product uses progress signals to make activity visible rather than leaving the user with raw task and tracking data.

The documented productivity score is a 0–100 composite based on task completion rate (40%), prayer consistency (30%), and health tracking (30%). Achievement milestones cover task activity, prayer streaks, and health logging.

Domain	Examples documented by the repository
Task	First Step · Getting Started · Task Master · Century Club · Productivity Hero
Prayer	Devotee · Consistent · Champion
Health	Health Tracker · Week Warrior · Health Champion · Hydration Hero

10 / CASE STUDY

Backend Architecture

The backend is structured into clear layers so that API concerns, AI orchestration, business logic and persistence do not collapse into one code path.

Layer	Responsibility
-------	----------------

FastAPI Application	HTTP routing, authentication, request/response handling
AI Agent Layer	Agent orchestration, model configuration and tool interaction
Tool Layer	Controlled actions exposed to the conversational agent
Service Layer	Business rules and application operations
Repository Layer	Generic and domain-specific data access
MongoDB + Beanie	Asynchronous document persistence
Scheduler / Notifications	Background jobs, FCM push and email delivery

11 / CASE STUDY

Backend Engineering Decisions

Async by design

MongoDB access is implemented through Beanie with asynchronous operations, matching FastAPI's asynchronous application model.

Repository pattern

Data access is separated from higher-level business logic, making persistence concerns easier to isolate and test.

Service layer

Application behavior is organized into services instead of embedding domain logic directly inside route handlers.

Validation & schemas

Pydantic schemas provide structured request/response validation around the API surface.

Authentication

JWT-based registration, login, refresh and logout flows provide the documented authentication foundation.

Middleware & error handling

The project structure includes middleware for logging, CORS and error handling.

12 / CASE STUDY

Notifications & External Integrations

The system includes both FCM push notifications and email delivery. This creates an output layer for the scheduled jobs and application events rather than relying solely on users opening the application.

The repository documents environment configuration for MongoDB, JWT signing, Mistral, Firebase credentials, Resend email delivery, and scheduler control. The project also includes Docker, docker-compose, Nginx configuration and Railway deployment configuration.

13 / CASE STUDY

Testing & Reliability Signals

The repository documents a 24-test suite covering authentication, tasks, health, prayer and analytics modules, with the documented status of 24/24 tests passing.

Area	Coverage documented
------	---------------------

Authentication	Register / login / refresh and related auth behavior
Tasks	Task lifecycle and task operations
Health	Health logging and summary behavior
Prayer	Prayer completion and tracking
Analytics	Progress, streak and analytics behavior

14 / CASE STUDY

Engineering Challenges & Problem-Solving

Challenge 1 — Making AI useful instead of decorative. A conversational interface alone does not create an intelligent productivity system. The solution was to expose application capabilities through explicit tools so that the agent can connect intent to action.

Challenge 2 — Preserving deterministic application behavior. Tasks, health records and prayer state are application data. They need predictable backend behavior rather than being generated by the model. The layered architecture keeps tool execution and persistence inside the application.

Challenge 3 — Modeling task lifecycle. A simple status field can allow invalid transitions. The DFA approach defines allowed transitions and makes task state behavior explicit.

Challenge 4 — Coordinating scheduled behavior. Reminders, digests and missed-prayer processing require background execution. APScheduler provides a dedicated scheduling layer.

Challenge 5 — Combining multiple domains. Tasks, health, prayer, notifications and analytics have different data models and behaviors. Separating repositories, services and tools allows them to coexist without turning the agent into one large block of business logic.

15 / CASE STUDY

What This Project Demonstrates

TaskFlow AI demonstrates a product-to-engineering workflow rather than a single isolated API. The project starts from a broader productivity problem, expands the product model, and then translates that model into concrete backend architecture.

Dimension	What it demonstrates
Product thinking	Turning a basic task-management concept into a broader personal productivity system.
Agentic AI	Connecting an LLM to controlled tools so conversation can trigger real application behavior.
Backend engineering	FastAPI, async MongoDB/Beanie, JWT, repositories, services, schemas and middleware.
System design	Separating AI orchestration from deterministic business logic and persistence.
Automation	Scheduled reminders, reports, digests and cleanup.
Reliability mindset	State-machine constraints, validation, layered architecture and automated tests.

16 / CASE STUDY

Key Learnings

- AI products become more useful when models are connected to real capabilities instead of being isolated chat interfaces.
- The model should interpret intent; application services should enforce business rules and state.
- Domain modeling matters even in productivity products. A DFA can make task behavior more predictable than free-form status updates.
- Background automation changes a product from a passive database into an active system that can remind, summarize and process events.
- A clean separation between routers, tools, services and repositories makes it easier to extend an AI system without rewriting its core.

17 / CASE STUDY

Future Direction

The repository explicitly identifies several roadmap items. These are presented as future direction rather than completed functionality:

- Push notification analytics dashboard
- Multi-language support for the AI agent
- Habit-building capabilities beyond prayer and health
- Public REST API rate-limiting tiers

18 / CASE STUDY

Technology Stack

Category	Technology
Backend	Python · FastAPI · Uvicorn
Database	MongoDB · Beanie async ODM
AI	OpenAI Agent SDK pattern · Mistral Large
Architecture	Routers · Tools · Services · Repositories
Authentication	JWT
Notifications	Firebase Cloud Messaging · Email / Resend
Scheduling	APScheduler
Validation	Pydantic
Testing	Pytest
Infrastructure	Docker · Docker Compose · Nginx · Railway configuration

19 / CASE STUDY

My Role

Built as an AI/backend engineering project, the work covered product concept development, backend architecture, AI-agent integration, tool design, data modeling, task-state logic, health and prayer modules, notifications, scheduled automation, analytics, testing and deployment-oriented infrastructure.

The project reflects the role I want my portfolio to communicate: **Agentic AI Engineer + Python Backend Engineer**—someone who can connect AI models to real application systems rather than treating AI as a standalone chat feature.

20 / CASE STUDY

Final Takeaway

TaskFlow AI began with a simple observation: task management can be useful, but productivity is larger than a checklist. The resulting system combines task workflows with health and prayer tracking, automated notifications, analytics and a conversational AI layer.

Its strongest technical idea is the bridge between language and execution: the user communicates naturally, the agent selects an approved tool, the service layer applies business logic, and the database remains the source of persistent state.

That architecture turns TaskFlow AI from a conventional task API into an exploration of what an AI-assisted personal productivity system can look like when conversational intelligence is connected to real backend capabilities.

TASKFLOW AI

AI-assisted productivity, health & prayer tracking — engineered around conversation, controlled actions and structured backend state.

Repository: github.com/abizaid24/TaskFlow-AI-Backend